

Technical Addendum: Findings from Initial Exploration and Revised Plan

Companion to *Methodology and Plan* (May 4, 2026)

Jun Li (Principal Investigator, PI; jun.li@nd.edu)

May 11, 2026

1 Purpose of This Document

This addendum records a sequence of technical findings that emerged from early exploration of the `rpart` conversion and from systematic consideration of issues that apply uniformly across any R-to-Python statistical-package conversion. The findings led to three substantive revisions to the original plan documented in *methodology_and_plan_Jun.pdf*:

1. A re-characterization of the compiled-code wrapping phase to account for the fact that R-package C and Fortran code is not portable standalone code but rather *R-extension code* that depends on R’s runtime, requiring an explicit **R-API extraction** step before Python binding can occur.
2. The introduction of a **compatibility shim** pattern that substantially reduces the cost of R-API extraction, allowing most algorithmic source files to compile unchanged.
3. A revised sequencing decision: **convert KernSmooth first, then rpart**, validating the methodology on a smaller and structurally different package before committing to the larger `rpart` effort. This yields two case studies for the methodology paper instead of one and produces a faster initial publishable artifact.

A fourth finding, less surprising but equally important, concerns the treatment of R’s base `stats` package functions, which appear pervasively in essentially every R package and need a uniform mapping strategy to Python equivalents. This is addressed in Section 4.

The findings here should be read as superseding the corresponding material in the main brief. The main brief’s overall structure, governance arrangements, required practices, and student-role specification remain valid; the technical sections on compiled-code wrapping, sequencing, and base-R function handling are refined here.

2 The R-Extension C Problem

2.1 What Was Discovered

Direct inspection of the `rpart` source (`src/rpart.c` and its companions) revealed that the C code in CRAN packages is generally *not* portable, standalone C. It is *R-extension C*: every exported entry point and most internal helpers depend on R’s C application programming interface, including:

- R’s tagged-pointer object type `SEXP`;
- the garbage-collection root-protection macros `PROTECT` and `UNPROTECT`;
- R’s allocation primitives `allocVector`, `allocMatrix`, and the transient allocator `R_alloc` (often used via the macro `ALLOC`);
- the `SEXP`-accessor macros `INTEGER`, `REAL`, `asInteger`, `asReal`;
- the list-construction API `SET_VECTOR_ELT`, `SET_STRING_ELT`, `mkChar`, `setAttrib`;
- R’s error and printing facilities `error()`, `Rprintf`, `REprintf`, along with the gettext translation macro `_(...)`.

These are calls into R’s runtime, not calls into a portable C library. The naive plan described in the original brief — “wrap the five `.Call()` entry points with `pybind11`” — is correct in intent but skips the prerequisite step: those entry points cannot be invoked from Python without R’s runtime present, because Python cannot construct `SEXP` objects, cannot satisfy `PROTECT`/`UNPROTECT`, and cannot run `R_alloc`.

Embedding R itself into the Python package is a non-starter: it would require shipping R alongside every wheel, defeating the central goal of producing a standalone, native Python package.

2.2 Two Categories of R-API Calls

A careful reading of the `rpart` sources shows that the R-API calls divide cleanly into two categories with very different treatment costs.

Category 1 — Pervasive but semantically simple. These appear throughout the algorithmic source files (in `rpart`’s case: `partition.c`, `bsplit.c`, `gini.c`, `anova.c`, `poisson.c`, `surrogate.c`, `xval.c`, and others) but are wrappers around operations that have plain-C equivalents:

R-API call	Semantics	Plain-C replacement
<code>R_alloc(n, size)</code> , <code>ALLOC</code>	transient allocation	<code>malloc</code> + arena
<code>error("msg")</code>	throw R error	callback / <code>longjmp</code>
<code>Rprintf</code> , <code>REprintf</code>	console output	<code>printf</code> , <code>fprintf</code>
<code>R_FINITE(x)</code>	NaN/Inf check	<code>isfinite(x)</code>
<code>_("msg")</code>	gettext translation	identity macro

Category 2 — Boundary-only, requires real refactoring. These appear only in the entry-point source files (in `rpart`’s case: `rpart.c`, `xpred.c`, `pred_rpart.c`, `rpartexp2.c`, `rpart_callback.c`). They are concerned with R’s object representation itself and cannot be transparently shimmed:

R-API element	Role
<code>SEXP</code> type	R’s tagged object pointer
<code>PROTECT</code> , <code>UNPROTECT</code>	GC root protection
<code>allocVector(INTSXP, n)</code> etc.	construct R-side return objects
<code>INTEGER(x)</code> , <code>REAL(x)</code>	accessor macros on <code>SEXP</code> inputs
<code>SET_VECTOR_ELT</code> , <code>mkChar</code> , <code>setAttrib</code>	list-construction
<code>R_NilValue</code>	R’s <code>NULL</code>

In rpart, the boundary code is concentrated in roughly 1,000 of the 4,454 total C lines; the remaining ~3,500 lines are Category-1 algorithmic code that becomes shim-able.

2.3 The Compatibility Shim Approach

The decisive insight: Category-1 calls can be handled by a single **compatibility header** that **#defines** each R-API symbol to its plain-C equivalent. Once this header is in place and included by the algorithmic source files, those files compile unchanged. The boundary entry points still need real refactoring, but the bulk of the codebase does not.

A skeleton of the shim header (`r_shim.h`):

```
/* r_shim.h --- drop-in replacement for R's C API in standalone builds */
#ifndef R_SHIM_H
#define R_SHIM_H

#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <setjmp.h>

/* Memory: arena allocator mimicking R_alloc transient semantics */
void *rshim_arena_alloc(size_t n, size_t size);
void rshim_arena_reset(void);
#define ALLOC(n, size) rshim_arena_alloc((n), (size))
#define R_alloc(n, size) rshim_arena_alloc((n), (size))

/* Errors: longjmp out to entry-point wrapper, converted to Python exception */
extern jmp_buf rshim_error_jmp;
extern char rshim_error_msg[1024];
void rshim_error(const char *fmt, ...);
#define error(...) rshim_error(__VA_ARGS__)
#define _(x) (x) /* no gettext */

/* Diagnostic output */
#define Rprintf printf
#define REprintf(...) fprintf(stderr, __VA_ARGS__)

/* Finite check */
#define R_FINITE(x) isfinite(x)

#endif
```

The corresponding `r_shim.c` implements the arena allocator (a list of `malloc` pointers, batch-freed at end of each entry-point call), the error function (`vsnprintf` followed by `longjmp`), and the buffer for the error message.

Each refactored entry-point wrapper then takes the form:

```
py::array_t<double> rpart_entry(/* plain C/numpy inputs */) {
    if (setjmp(rshim_error_jmp) == 0) {
        rshim_arena_reset();
        /* ... call the algorithmic core ... */
        /* ... convert plain-C outputs to numpy arrays ... */
        return result;
    }
}
```

```

    } else {
        throw std::runtime_error(rshim_error_msg);
    }
}

```

This pattern is well-established (used by GSL, by parts of LAPACK, and by other scientific C libraries with their own error mechanisms). It provides the semantic equivalent of R’s transient allocation and abortive error model while remaining inside standard C/C++.

2.4 Revised Phase 2 of the Methodology

The original Phase 2 (“Wrapping Compiled Code”) becomes a two-step process:

- **Phase 2a — R-API extraction.** Author the shim header `r_shim.h` (a one-day task) and refactor the entry-point source files to take plain C/C++ inputs and return plain C/C++ structures, with errors routed through `setjmp/longjmp`. The number of entry-point files is small (five in `rpart`’s case), so this is bounded work measured in weeks, not months.
- **Phase 2b — Python binding.** Wrap the refactored entry points with `pybind11` (or `ffi` for pure-C codebases where C++ is unwanted). The arena allocator and the `setjmp/longjmp` error path live entirely in the binding layer; the algorithmic code remains unchanged.

The total Phase 2 effort, with the shim approach, is approximately 4–5 weeks for `rpart`-scale packages — only modestly more than the naive plan and substantially less than per-call refactoring would have required.

2.5 Generality

The shim approach is not specific to `rpart`. *Every* R package with compiled code uses the same R-API surface; the same `r_shim.h` works, with at most minor additions, for any package the program later converts. The shim is itself a reusable artifact and a candidate standalone deliverable (a small `rshim` library on PyPI, intended for use by anyone converting an R-extension C codebase to a standalone Python extension).

3 Re-Sequencing: KernSmooth First, Then rpart

3.1 Why Re-Sequence

The shim approach is the right strategy but is unproven. Validating it directly on `rpart` would risk debugging methodology problems against 4,454 lines of C, which is a costly substrate for diagnosis. A smaller package, structurally different from `rpart`, lets the shim approach get debugged at small scale and produces a complete, publishable artifact substantially faster than the `rpart`-first plan would.

There are three additional benefits:

1. **The methodology paper becomes stronger with two case studies than with one.** A single case study leaves reviewers asking “does this generalize?” Two structurally different case studies preempt that question.
2. **Reusable infrastructure built on the smaller package accelerates the larger one.** The build system, the fixture framework, the shim header, the first few Claude Code Skills,

the upstream-sync pipeline — all are produced once on the smaller package and reused on rpart.

3. **An early publishable artifact** (PyPI release + JOSS paper for the smaller package) provides momentum and a citable credibility signal before rpart is complete.

3.2 KernSmooth as the Initial Target

Why KernSmooth. The package `KernSmooth` (M.P. Wand, with Brian Ripley as the R port maintainer) implements binned kernel density estimation (`bkde`, `bkde2D`), local polynomial regression (`locpoly`), and plug-in bandwidth selection (`dpik`, `dpill`). It satisfies all the selection criteria established in the main brief:

- **Zero external R-package dependencies** (only base R).
- **Recommended-package status** (ships with R itself).
- **Compact compiled-code surface** (~700 lines of Fortran).
- **Compact R surface** (~600 lines, a handful of functions).
- **Recognized author pedigree.** Wand co-authored the standard textbook *Kernel Smoothing* (1995); Ripley is on the R Core team.
- **Genuine Python ecosystem gap.** While `scipy.stats.gaussian_kde` and several Python KDE libraries exist, *plug-in bandwidth selectors* (Sheather–Jones-style direct plug-in for univariate and bivariate cases) and *locpoly* (local polynomial regression with binning) are absent or weak in the Python stack. Wand’s plug-in methods are the gold-standard reference for bandwidth selection and have no faithful Python implementation.
- **Permissive license** (“Unlimited”; more permissive than GPL).

Popularity statistics (May 2026).

Metric	Value
Downloads, last 7 days (RStudio CRAN mirror)	13,357
Downloads, last 30 days	45,459
Downloads, last 12 months	556,692
Reverse depends on CRAN	17
Reverse imports on CRAN	90
Reverse suggests on CRAN	46
Total reverse-dependency count	153
CRAN priority	recommended
Latest version	2.23-26 (Jan. 2025)

For comparison, rpart’s last-12-month total is 1,005,525 downloads, so KernSmooth runs at approximately 55% of rpart’s volume. KernSmooth is roughly six times as popular as `acepack` (91,389 downloads), the only seriously smaller candidate considered. KernSmooth is therefore unambiguously a top-tier package in its category, not a toy.

Honest qualifications. Two qualifications deserve explicit note:

- **KernSmooth is more slowly maintained than rpart.** The last upstream release (2.23-26) is from January 2025. The package is mature and stable rather than actively evolving. This is largely a feature for our purposes (no moving target during conversion), but the upstream-sync pipeline demonstration is less compelling on KernSmooth alone.
- **The Python gap is narrower than for rpart.** The targeted-user case is “I want Sheather–Jones bandwidth selection in Python; I want `locpoly`” rather than “I want KDE.” This is a narrower but real audience. The methodology paper’s two-case-study structure should explicitly cover this asymmetry: KernSmooth validates the framework on a focused-gap package; rpart validates it on a broad-gap package.

Fortran versus C. KernSmooth’s compiled code is Fortran; rpart’s is C. This is a *methodological feature*, not a bug. Validating the shim approach on both compiled languages strengthens the methodology paper’s generality claim. Many of the highest-value remaining R packages (`quantreg`, `acepack`, `logspline`, `robustbase` core) are Fortran-based; doing KernSmooth first establishes that the framework handles them.

For Fortran wrapping, the recommended Python tool is `f2py` (part of NumPy), which auto-generates Python bindings from Fortran sources with relatively simple annotation. The shim concept extends to Fortran with minor adjustments: most R-package Fortran code operates on plain `REAL*8` and `INTEGER` arrays and does not call back into R from inside Fortran, so the boundary code in this case is at the `.Fortran()` call site in R, not inside the Fortran sources themselves. This typically makes Fortran wrapping *easier* than C wrapping, not harder.

3.3 Revised Sequencing

The revised plan is:

- **Weeks 1–10 — KernSmooth conversion.** Builds the shim infrastructure (both `r_shim.h` for any future C code and a parallel Fortran-side shim where needed), the fixture framework, the first Claude Code Skills, and a working release-ready package.
- **Weeks 11–14 — KernSmooth polish and release.** PyPI release, JOSS submission, ReadTheDocs site, first difficulty-log and decision-log entries published as supplementary material.
- **Weeks 15–32 — rpart conversion.** Reuses the validated infrastructure. Probably faster than the original 18-week estimate because much groundwork is complete.
- **Weeks 33+ — methodology paper drafted with both case studies**, plus optional further validation on a third small package.

Total project length: approximately eight months for a substantially stronger end state than the original 4.5–5 months would have produced.

3.4 Implications for the Methodology Paper

The paper’s central claim becomes:

We validate the methodology on two structurally different R packages: KernSmooth (Recommended-status, Fortran compiled code, kernel-smoothing methods, narrow-gap audience) and rpart (Recommended-status, C compiled code, recursive partitioning, broad-gap audience). Both ports

achieve full feature parity with their R counterparts and are released on PyPI under permissive distribution.

This is a meaningfully stronger framing than the single-case-study version. It also opens the path to follow-on conversions (`quantreg`, `logspline`, `acepack`, robust statistics, sandwich estimators, etc.) as natural extensions of an already-validated framework.

4 Handling R stats Function Dependencies

4.1 The General Picture

Every R package, including `KernSmooth` and `rpart`, uses functions from R’s base `stats` package: distribution functions, optimization, linear-algebra solvers, formula handling, NA semantics, density estimation, and so on. A uniform mapping strategy to Python equivalents is required across the program.

The good news is that **the great majority of stats functions have direct or near-direct Python equivalents** in `numpy`, `scipy`, `statsmodels`, and `pandas`. Translation for this majority is a mapping exercise and is well within the capability of AI-assisted conversion.

The bad news is that **a small number of stats elements have no clean Python equivalent** or have Python equivalents with subtly different semantics, and these need careful, centralized handling.

4.2 Direct-Equivalent Functions

The following representative mapping illustrates the easy 80% of `stats` translation. Yufei will expand this table as the project’s actual catalog of used functions grows.

R stats	Python equivalent	Notes
<code>dnorm</code> , <code>pnorm</code> , <code>qnorm</code> , <code>rnorm</code>	<code>scipy.stats.norm.{pdf,cdf,ppf,rvs}</code>	Pattern generalizes
<code>pchisq</code> , <code>pf</code> , <code>pt</code> , <code>qt</code>	<code>scipy.stats.{chi2,f,t}.{cdf,ppf}</code>	Same pattern
<code>lm.fit</code> , <code>lm.wfit</code> , <code>lsfit</code>	<code>numpy.linalg.lstsq</code> , <code>scipy.linalg.lstsq</code>	QR-based
<code>qr</code> , <code>chol</code> , <code>solve</code>	<code>numpy.linalg.{qr,cholesky,solve}</code>	Direct
<code>optim</code> (BFGS)	<code>scipy.optimize.minimize(method='BFGS')</code>	See gotcha
<code>nlminb</code>	<code>scipy.optimize.minimize(method='L-BFGS-B')</code>	Approximate
<code>uniroot</code>	<code>scipy.optimize.brentq</code>	Direct
<code>optimize</code>	<code>scipy.optimize.minimize_scalar</code>	Direct
<code>approx</code> , <code>approxfun</code>	<code>scipy.interpolate.interp1d</code>	Direct
<code>splinefun</code>	<code>scipy.interpolate.UnivariateSpline</code>	Direct
<code>density</code>	<code>scipy.stats.gaussian_kde</code>	See gotcha
<code>median</code> , <code>var</code> , <code>sd</code> , <code>cor</code>	<code>numpy.{median,var,std,corrcoef}</code>	Watch <code>ddof</code>
<code>quantile</code>	<code>numpy.quantile</code>	See gotcha
<code>t.test</code> , <code>wilcox.test</code>	<code>scipy.stats.{ttest_ind,chisquare,wilcoxon}</code>	Direct
<code>na.omit</code>	<code>pandas.DataFrame.dropna</code> , masked indexing	Direct
<code>setNames</code>	dict comprehension, <code>pandas.Series</code>	Idiom
<code>tabulate</code> , <code>table</code>	<code>numpy.bincount</code> , <code>pandas.value_counts</code>	Direct
<code>cumsum</code> , <code>cumprod</code>	<code>numpy.{cumsum,cumprod}</code>	Direct

4.3 Hard Cases

A small number of `stats` elements need explicit, centralized handling. Each is addressed once and the resolution is reused across all converted packages.

Formula and model-frame machinery. R packages routinely accept formulas ($y \sim x1 + x2 * x3$) and use `model.frame()`, `model.matrix()`, `model.response()`, `model.weights()`, `model.offset()`, `terms()`, and `na.action` to extract design matrices, responses, and weights, with implicit factor encoding (treatment coding by default).

Four reasonable strategies exist:

1. **Use formulaic** (Matthew Wardrop’s modern Python formula library; successor to `patsy`, which `statsmodels` is deprecating). Actively maintained and produces design matrices comparable to R’s `model.matrix`.
2. **Use patsy** (older library, mature but stagnant).
3. **Roll a minimal formula helper** tailored to the small subset of formula semantics each package actually uses.
4. **Skip formulas entirely.** Make the primary API matrix-based (`X`, `y`) and treat formulas as an optional convenience layer.

Recommended pattern: strategy (4) as the primary API plus strategy (1) as an optional formula adapter. This matches modern scientific Python practice (scikit-learn is matrix-first), avoids fragility around formula edge cases, and keeps the package’s core API independent of an external formula library.

quantile — nine types in R, one default in NumPy. R’s `quantile(x, p)` accepts a `type` parameter from 1 to 9 (Hyndman & Fan 1996), with R’s default being `type = 7` (linear interpolation). `numpy.quantile`’s current default `method='linear'` corresponds to R’s `type = 7`, but older NumPy versions used a different default and many third-party libraries pass through unspecified defaults that may not match.

Resolution: explicitly specify `method='linear'` in NumPy calls when porting R’s `quantile()`. Document that the converted package matches R’s `type = 7` default. Provide a documented escape hatch if a different type is needed.

optim — equivalent algorithms with different tolerances. R’s `optim(method = "BFGS")` and `scipy.optimize.minimize(method = "BFGS")` both implement BFGS but with different default tolerances, line-search strategies, and convergence criteria. Numerical results will be close but not bit-identical.

Resolution: for fidelity testing, accept “equivalent quality of optimum” rather than “bit-identical results.” Document the difference and the choice in the decision log. Where bit-identical results are required for testing, implement R’s exact algorithm (rare; should be the exception).

density — different default bandwidths. R’s `density()` defaults to `bw = "nrd0"` (Silverman’s rule); `scipy.stats.gaussian_kde` defaults to Scott’s rule. These produce slightly different bandwidths and therefore slightly different KDEs.

Resolution: document the difference, accept an explicit `bw.method` argument, and for fidelity testing use the same bandwidth on both sides.

NA semantics and `na.action`. R has a unified NA system propagating through most operations with behavior controlled by `na.action`. Python uses `numpy.nan` for floats (well-defined) and `pandas.NA` for nullable integer/boolean (newer, less universally supported).

Resolution: accept `numpy.nan` as the NA representation; document policy; provide options `na_action="omit", "propagate", "error"` mirroring R’s defaults. For `rpart` specifically, NAs route to the surrogate-split mechanism, which is the package’s signature feature.

Random-number reproducibility. R’s RNGs (`set.seed()`) and Python’s RNGs are not interchangeable. Identical seeds do not produce identical random streams across languages.

Resolution: do not attempt cross-language seed-identical reproducibility. Document this explicitly. For testing involving randomness, either compare statistical properties (mean, variance, distributional fit) rather than exact values, or supply both sides with externally-generated random vectors.

4.4 The `r_stats_compat` Module

In parallel with `r_shim.h` for C-API translation, the methodology calls for an `r_stats_compat.py` module that centralizes the translation of R `stats` calls to Python. This module contains:

1. A reference **translation table** mapping R `stats` functions to Python equivalents, with notes on semantic differences. (The table is also methodology-paper appendix material.)
2. **Thin wrappers** for cases where defaults differ — e.g., `quantile_r7(x, q)` calling `numpy.quantile(x, q, method='linear')`, making R semantics explicit at call sites.
3. A **formula adapter** wrapping `formulaic` for the formula subset packages actually need, with documented limitations.
4. An **NA-action helper** standardizing NA policy across the package family.

This module is built once on `KernSmooth`, refined during `rpart`, and reused for every subsequent conversion. It is itself a publishable artifact — a small “R-stats compatibility layer for Python” package.

4.5 Process Recommendation

Before substantive translation begins on either `KernSmooth` or `rpart`, Yufei should produce an explicit **catalog** of all `stats` function calls in the package, for example:

```
grep -hE '\b(model\.(frame|matrix|response|weights|offset)|terms|formula|
na\.(omit|action|pass)|lm\.(w?)fit|lsfit|optim|nlminb|uniroot|
optimize|approx(fun)?|splinefun|density|quantile|median|var|
sd|cor|t\.test|chisq\.test|wilcox\.test|setNames|relevel|
tabulate|cumsum|cumprod|dnorm|pnorm|qnorm|rnorm|pchisq|
qchisq|pf|qf|pt|qt|rt|runif)\b' R/*.R | sort -u
```

This produces the full list of base-R **stats** (and related base) calls to handle. For KernSmooth the list will be short (likely under 20 distinct functions); for rpart it is longer but still bounded. The catalog seeds `r_stats_compat.py`.

5 Implications for the Methodology Framework

The findings above necessitate three explicit additions to the methodology framework as described in the main brief.

5.1 Updated Phase Structure

The nine-phase framework in the main brief becomes a ten-phase framework with the following modifications:

- **Phase 2a (new)**: R-API extraction via the compatibility-shim approach.
- **Phase 2b (refined)**: Python binding of the R-API-free entry points.
- **Phase 4.5 (new)**: **stats**-call cataloging and construction of `r_stats_compat.py`; this phase runs in parallel with orchestration conversion (Phase 4) and feeds back into it.

The rest of the framework (reconnaissance, fixture-based testing, orchestration conversion, R–Python semantic adjustments, documentation conversion, distribution, upstream synchronization, discrepancy auditing) is unchanged.

5.2 New Reusable Artifacts

Two reusable infrastructure artifacts emerge from the methodology that did not exist in the original framework:

1. `r_shim.h` (and supporting `r_shim.c`): the compatibility header for R-extension C compiled code. Potentially distributable as a standalone PyPI package (working name: `rshim`).
2. `r_stats_compat.py`: the centralized R-**stats** translation module with reference table, thin wrappers, formula adapter, and NA-policy helper. Also potentially a standalone PyPI package (working name: `r-stats-compat` or `rstats-py`).

These artifacts are themselves cited contributions of the methodology program and should be packaged, versioned, and released independently. They are exactly the kind of reusable cyberinfrastructure that strengthens both the methodology paper and any future funding proposal.

5.3 New Skill Suite Additions

The eight-skill Claude Code Skill suite described in the main brief expands to ten:

1. `r-api-shim-builder` — given a compiled-code source tree, identifies R-API calls, categorizes them, and produces a draft `r_shim.h`.
2. `r-stats-call-cataloger` — scans R source for **stats**-package calls, classifies each as direct-equivalent / wrapper-needed / hard-case, and seeds the package’s `r_stats_compat.py`.

The other eight skills (recon, planner, compiled-code-wrapper, fixture-generator, orchestration-converter, discrepancy-debugger, rd-to-numpy-docstring, upstream-sync-setup) remain as originally described.

5.4 Updated Timeline

Total project length is revised from approximately 4.5–5 months (rpart-only) to approximately eight months (KernSmooth then rpart), with the following coarse decomposition:

- Weeks 1–10: KernSmooth conversion (including shim development and `r_stats_compat.py` bootstrap).
- Weeks 11–14: KernSmooth polish, PyPI release, JOSS paper.
- Weeks 15–32: rpart conversion.
- Weeks 33–38: rpart polish, second JOSS paper, methodology paper drafted with both case studies.

Within KernSmooth itself, the first two weeks remain infrastructure (repository setup, build system, hello-world wrap), then weeks 3–4 build the shim and refactor the first entry point, weeks 5–8 do the bulk of the conversion, and weeks 9–10 finalize tests and documentation.

Closing Note

The findings recorded here are exactly the kind of early-surfaced methodology insight the original brief's discipline of difficulty logging was designed to produce. They illustrate the value of the discipline: the R-extension C issue and the categorization that led to the shim approach were not visible in the original plan but became clear within days of substantive engagement with the source. The pivot to KernSmooth-first and the centralized handling of base-R `stats` calls follow naturally from those technical findings.

The main brief's methodological substance — the layered architecture, the fixture-based testing discipline, the operationalization via Claude Code Skills, the difficulty-and-decision log practices, and the publication strategy — remains unchanged. What this addendum updates are the specific technical phases and the sequencing of case-study packages. The contribution becomes stronger, not weaker, with these revisions: a two-case-study methodology paper covering both Fortran and C compiled code, with reusable infrastructure deliverables (the shim header and the stats compatibility module) that did not exist in the original plan.

The student should treat this addendum as authoritative supplement to the main brief, on all matters where the two overlap, the addendum supersedes.

End of addendum.